

EDYGUIDE

AI-Driven Smart Student Service Ecosystem

System Design & Technical Specification Document

Version 1.0 | B2B Edition | Egypt Higher Education Market

Project Summary

EDYGUIDE is a comprehensive AI-powered student service ecosystem specifically designed for Egyptian universities. It uniquely combines proactive academic support (SVM-based nudging), confidential Arabic mental health assistance (Saleem model), and LLM-powered career coaching (Llama-3 CV analysis + Wuzzuf job matching). Built on Flutter + FastAPI + Firebase, it is fully compliant with Egypt's Law 151/2020 and designed to scale to 20,000+ concurrent users. The projected outcome is a 21% increase in student service engagement and a measurable improvement in retention rates through early at-risk identification.

Section 1: Platform Services

EDYGUIDE consolidates all student-facing services into a single AI-powered super-app. The following services are organized into two tiers: standard administrative services that represent baseline market expectations, and innovative value-add services that differentiate the platform.

1.1 Standard Administrative Services

These services match current market expectations in Egyptian universities and form the core reliability layer of the platform.

S1	Admissions & Enrollment Help Answers questions about application fees, required documents, and admission deadlines. Automates repetitive inquiries to reduce administrative load.
S2	FAQ Automation Handles high-volume repetitive queries about campus locations, office hours, and university policies using a rule-based + LLM hybrid engine.
S3	Academic Records Access Integrates with university SIS (e.g., Banner) to allow students to view grades, GPA history, and attendance records in real time.
S4	Schedule & Timetable Management Displays daily and weekly lecture schedules, exam timetables, and room assignments. Syncs with LMS platforms such as Moodle or Canvas.

PROJECT : Student Service Chatbot

S5

IT Support

Assists with password resets, Wi-Fi access issues, and Learning Management System troubleshooting without requiring a visit to the IT office.

S6

General Campus Information

Provides directions to buildings, library hours, faculty contact details, and campus event listings.

1.2 Innovative Value-Add Services

These services address critical gaps in the Egyptian higher-education market and represent the primary competitive differentiators of EDYGUIDE.

A. Academic Concierge

S7

AI Micro-Tutoring

Provides instant explanations of course concepts drawn from the university's specific syllabus, available in both Modern Standard Arabic and Egyptian dialect.

B. Mental Health & Well-being

S8

Arabic Mental Health Chatbot

Offers confidential, anonymous mental health check-ins, mood tracking, and guided mindfulness sessions. Built on NLP tuned to Egyptian Arabic dialect and cultural context. Includes a 'Warm Handoff' protocol that escalates to a live counselor when distress signals are detected.

C. Career & Internship Bridge

S9

CV / ATS Optimizer

Students upload their CV and receive an instant ATS-compatibility score benchmarked against Egyptian startup and corporate standards, powered by Llama-3 via Watsonx.ai.

S10

Smart Career & Internship Matching

Analyzes a student's grades, skills, and interests to recommend specific internship openings at relevant Egyptian companies (cross-referenced with Wuzzuf and LinkedIn postings).

PROJECT : Student Service Chatbot

S11

Mock Interview Simulator

Conducts text-based practice interviews for specific roles (e.g., Junior Accountant at an Egyptian bank) and provides instant feedback on clarity, tone, and technical accuracy.

E. Social & Peer Learning

S12

Peer-to-Peer Textbook Marketplace

A bot-managed internal marketplace for Download used PDFs and lab equipment within the campus community.

PROJECT : Student Service Chatbot

Section 2: B2B Prototype Explanation

EDYGUIDE is designed as a Business-to-Business (B2B) platform. The end-users are university students, but the paying client is the university institution itself. This section explains the B2B rationale and how a working prototype can be demonstrated to prospective institutional clients.

2.1 What B2B Means for EDYGUIDE

In a B2B model, EDYGUIDE is licensed or sold to a university as an institutional product. The university then deploys it to its student body. This distinction has several important implications:

Client	The university administration, IT department, and student affairs office — not individual students.
Revenue Model	Annual licensing fee, per-student seat pricing, or a tiered SaaS subscription based on feature modules activated.
Integration	EDYGUIDE integrates with the university's existing SIS (e.g., Banner, PeopleSoft), LMS (Moodle, Canvas), and payment infrastructure.
Data Ownership	All student data remains the property of the university and is stored in compliance with Egypt's Law No. 151 of 2020.
Customization	Each university deployment can be white-labeled with the institution's branding, colors, and logo.

PROJECT : Student Service Chatbot

Section 3: Website Screens & Buttons

This section documents every screen, its purpose, and all interactive elements (buttons, inputs, navigation links) that must be implemented. This serves as the UI specification for the frontend development team.

3.1 Public-Facing Screens (Pre-Login)

Screen 1: Welcome / Landing Page

The entry point for the platform. Presents the EDYGUIDE brand and directs visitors to sign in or learn more.

Buttons & Elements:

- Get Started / Sign In Button — navigates to the Login screen.
- Learn More Button — smooth-scrolls to the About section.
- Navigation Bar — links: Home, About, Services, Contact.
- University Logo Placeholder — customizable per institutional deployment.

Screen 2: About Page

Explains the platform value proposition to prospective students and university administrators.

Buttons & Elements:

- Back to Home Button — returns to the landing page.
- Sign In Button — persistent CTA in the navigation bar.
- Feature Highlights Cards — non-interactive display cards for each module (Academic, Wellness, Career, Finance).

Screen 3: Sign In / Login Page

Secure authentication gate using the university's Single Sign-On (SSO) credentials.

Buttons & Elements:

- University Email Input Field — accepts the institutional email format only.
- Password Input Field — masked input with show/hide toggle.
- Login Button — submits credentials; triggers MFA flow.
- MFA Code Input Field — appears after successful credential verification.
- Verify Button — submits MFA code.
- Forgot Password Link — routes to the university IT support or password reset portal.
- SSO via Google / Microsoft Button — optional federated login.

3.2 Core Application Screens (Post-Login)

PROJECT : Student Service Chatbot

Screen 4: Home Dashboard

The student's personalized control tower. The first screen seen after login.

Buttons & Elements:

- Daily Nudge Card — dismissable notification with 'View Details' link.
- Quick Access Tiles — clickable tiles for: My Grades, My Schedule, Pay Fees, Chat with Concierge.
- Notification Bell Icon — opens a dropdown of recent alerts.
- Profile Avatar / Menu — opens profile settings, logout option.
- Bottom Navigation Bar — tabs: Home, Concierge, Wellness, Career, My Account.

Screen 5: Academic Levels Screen

Displays the student's academic journey organized by year (Level 1 through Level 4).

Buttons & Elements:

- Level Cards (1-4) — clickable to expand level-specific course list.
- View Credits Button — shows credit hour progress toward graduation.
- Graduation Tracker Button — opens the predictive graduation timeline.

Screen 6: Level Content / Course Detail Screen

Shows course-specific details including materials, attendance, and AI advisor access.

Buttons & Elements:

- Course Material Button — opens LMS integration (Moodle/Canvas link).
- View Attendance Button — shows session-by-session attendance log.
- Ask AI Advisor Button — opens the concierge chat pre-loaded with course context.
- Back Arrow — returns to the Levels screen.

Screen 7: AI Model Concierge Chat Screen

The primary conversational interface. Students ask any academic, administrative, or informational question.

Buttons & Elements:

- Text Input Field — message composition area.
- Send Button — submits the message to the AI engine.
- Voice Input Button — optional speech-to-text input.
- Attachment Button — allows file uploads (CV, assignments) for analysis.
- Quick Reply Chips — pre-suggested options (e.g., 'Check my GPA', 'Pay a fee', 'Show schedule').
- Chat History Scroll — displays full conversation thread with timestamps.
- Clear Chat Button — resets the conversation.

PROJECT : Student Service Chatbot

Screen 8: Career Bridge Screen

The student's career development hub for CV review, job matching, and interview practice.

Buttons & Elements:

- Upload CV Button — opens file picker; accepts PDF and DOCX formats.
- Scan for ATS Button — initiates the LLM-powered ATS scoring engine.
- Optimize LinkedIn Profile Button — routes to the LinkedIn Profile Optimizer module.
- Find Internships Button — opens personalized internship recommendations.
- Start Mock Interview Button — launches the text-based interview simulator.
- Request Alumni Connection Button — triggers the Alumni Matchmaker flow.
- View Skill Gaps Button — displays missing skills based on target job postings.

Screen 9: Admin Dashboard (University Staff View)

A separate interface for university administrators to monitor platform usage and respond to requests.

Buttons & Elements:

- View Pending Requests Button — lists all student service requests awaiting response.
- Approve / Reject Request Buttons — action buttons on each request card.
- Export Analytics Button — downloads a usage and engagement report.
- Manage Scholarships Button — add, edit, or archive scholarship listings.
- Broadcast Notification Button — sends a message to all students or a specific group.

PROJECT : Student Service Chatbot

Section 4: Technical Stack — Frontend, Backend & Database

This section defines each layer of the EDYGUIDE technical architecture, including its components, definition, and a balanced assessment of advantages and disadvantages based on the chosen implementation.

4.1 Frontend

Definition:

The frontend is the client-side layer of the application — everything the student or administrator sees and interacts with directly. It runs on the user's device (mobile phone or browser) and is responsible for rendering the user interface, handling user inputs, and communicating with the backend via API calls.

Chosen Technology: Flutter (Dart)

Flutter is Google's open-source UI toolkit that compiles to native Android, iOS, and web from a single codebase. EDYGUIDE uses Flutter for its cross-platform mobile app.

Core Components:

- **Widgets** — Flutter's building blocks. Every UI element (buttons, text, cards, inputs) is a widget. Stateful widgets hold dynamic data; stateless widgets are purely presentational.
- **Navigation & Routing** — Manages transitions between screens using Navigator 2.0 and `go_router` for deep-linking support.
- **State Management** — Provider or Riverpod is used to manage app-wide state (logged-in user data, chat history, payment status).
- **HTTP Client** — The `dio` package handles all REST API calls to the FastAPI backend, including token injection and error handling.
- **Local Storage** — `shared_preferences` stores lightweight settings (theme preference, cached user data) on the device.
- **Push Notifications** — `firebase_messaging` handles proactive nudge notifications sent from the backend.
- **Platform Channels** — Used to access device-specific features (camera for profile photo, biometric authentication for login security).

Aspect	Details
Advantage 1	Single codebase deploys to Android, iOS, and web — reduces development time and maintenance cost compared to maintaining separate native apps.
Advantage 2	Hot Reload feature dramatically speeds up the UI development cycle by reflecting code changes in real time without restarting the app.

PROJECT : Student Service Chatbot

Advantage 3	Pixel-perfect rendering using its own rendering engine (Impeller/Skia) ensures a consistent UI across all devices and screen sizes.
Advantage 4	Growing ecosystem with strong community support and first-party packages from Google (e.g., firebase_*, google_maps_flutter).
Disadvantage 1	Dart is not a widely-known language in the Egyptian tech talent market, which may make it harder to hire experienced Flutter developers locally.
Disadvantage 2	Flutter web support is functional but less mature than dedicated web frameworks like React. Complex web-specific behaviors (SEO, browser APIs) require workarounds.
Disadvantage 3	App bundle size is larger than native apps due to the embedded Flutter engine, which may be a concern for students on limited mobile data plans.
Disadvantage 4	Deep platform-specific integrations (e.g., complex Fawry SDK customizations) may require native Kotlin/Swift code via platform channels, adding complexity.

4.2 Backend

Definition:

The backend is the server-side layer responsible for processing business logic, handling authentication, communicating with the database, calling external APIs (Fawry, Vodafone Cash, LLM services), and responding to frontend requests. It acts as the brain of the application — invisible to the user but critical to every operation.

Chosen Technology: Python with FastAPI

FastAPI is a modern, high-performance Python web framework designed for building APIs. It is built on top of Starlette (ASGI) and Pydantic, making it ideal for handling concurrent AI inference requests.

Core Components:

- **API Router** — Defines all REST endpoints organized by domain (e.g., /academic, /payments, /career, /wellness).
- **Authentication Service** — Manages JWT token generation and validation. Enforces Multi-Factor Authentication (MFA) and integrates with university SSO via OAuth 2.0.
- **AI Orchestration Layer** — Routes student queries to the appropriate AI engine: rule-based responses for FAQs, Llama-3 via Watsonx.ai for career/CV analysis, and SVM models for attendance-based predictions.
- **SIS Integration Service** — Communicates with the university's Student Information System (Banner or equivalent) via secured REST or SOAP APIs to pull live grade, attendance, and schedule data.

PROJECT : Student Service Chatbot

- Payment Gateway Service — Wraps the FawryPay SDK and Vodafone Cash API to generate payment references, confirm transactions, and log payment records.
- Notification Scheduler — A background task runner (using APScheduler or Celery) that triggers proactive nudges based on SIS data events (e.g., attendance dropping below threshold).
- Document Workflow Engine — Tracks the status of administrative document requests by polling or subscribing to the university's internal workflow system.

Aspect	Details
Advantage 1	FastAPI is one of the fastest Python frameworks available, capable of handling thousands of concurrent requests — critical for an AI-heavy platform serving a large student body.
Advantage 2	Automatic interactive API documentation (Swagger UI and ReDoc) is generated from code annotations, making integration with the frontend team straightforward.
Advantage 3	Python's rich AI/ML ecosystem (scikit-learn for SVM, LangChain for LLM orchestration, transformers for NLP) integrates natively with the backend without inter-language bridges.
Advantage 4	Async/await support out of the box enables non-blocking I/O for external API calls (Fawry, Vodafone Cash, Watsonx.ai), improving throughput significantly.
Disadvantage 1	Python's Global Interpreter Lock (GIL) limits true CPU-bound parallelism. CPU-intensive model inference should be offloaded to separate worker processes or external inference endpoints.
Disadvantage 2	FastAPI requires more architectural discipline than opinionated frameworks like Django. Without clear structure, large codebases can become disorganized.
Disadvantage 3	Integrating with legacy university SIS systems (which may use SOAP or proprietary protocols) requires additional adapter development that can be time-consuming.
Disadvantage 4	Deployment and DevOps complexity is higher than managed backend platforms (e.g., Firebase Functions), requiring expertise in containerization (Docker) and orchestration (Kubernetes or equivalent).

4.3 Database (DB)

Definition:

The database is the persistent storage layer of the application. It stores all structured and semi-structured data including student profiles, chat histories, payment records, academic data

PROJECT : Student Service Chatbot

caches, career profiles, and wellness logs. The database must meet strict security, availability, and compliance requirements under Egypt's Law No. 151 of 2020 on Personal Data Protection.

Chosen Technology: Firebase Firestore (NoSQL Cloud Database)

Firebase Firestore is a fully managed, serverless, document-oriented NoSQL database provided by Google Cloud. It offers real-time synchronization, offline support, and built-in security rules.

Core Components:

- **Collections & Documents** — Data is stored as JSON-like documents organized into collections. Core collections include: students, chat_sessions, chat_messages, service_requests, payments, career_profiles, wellness_logs, scholarships.
- **Security Rules** — Firestore's built-in rule engine enforces row-level access control, ensuring that a student can only read their own data and that admin-only collections are protected.
- **Real-Time Listeners** — Clients subscribe to document changes and receive updates instantly without polling — used for live chat, payment status, and document tracking.
- **Composite Indexes** — Custom indexes are defined to support complex queries (e.g., 'all students in Department X with attendance below 75%').
- **Firebase Authentication Integration** — User identity from Firebase Auth is directly used in security rules to gate data access.
- **Cloud Storage Bucket** — Linked storage for file uploads (CVs, assignments, profile photos) with access tokens tied to student identity.
- **Backup & Export** — Scheduled Firestore exports to Google Cloud Storage provide data backup and enable compliance audit trails.

Aspect	Details
Advantage 1	Real-time synchronization means changes in the database (e.g., a counselor updating a student's wellness case) are reflected instantly in the student's app without requiring a page refresh or polling.
Advantage 2	Serverless and fully managed — no database administration overhead (patching, scaling, failover) required from the development team.
Advantage 3	Offline-first capability: Firestore SDK caches data locally, so students can view their schedules and recent chat history even without an internet connection.
Advantage 4	Seamless integration with the rest of the Firebase ecosystem (Authentication, Cloud Functions, Storage, Cloud Messaging) reduces the number of third-party services to manage.
Disadvantage 1	NoSQL document model does not support complex relational queries (JOINS). Data that has many-to-many relationships (e.g., tutors and students) requires careful data denormalization or client-side aggregation.
Disadvantage 2	Pricing is consumption-based (per read, write, and delete operation). At scale — with thousands of students triggering real-time listeners

PROJECT : Student Service Chatbot

	simultaneously — costs can escalate quickly without careful query optimization.
Disadvantage 3	Limited support for advanced analytics queries. Running aggregate reports (e.g., 'average GPA by department this semester') requires exporting data to BigQuery, adding architectural complexity.
Disadvantage 4	Data residency: Firebase's default data centers may not satisfy strict Egyptian data sovereignty requirements. Deploying to a Google Cloud region closest to Egypt (e.g., europe-west1) must be configured explicitly, and cloud-based storage may face institutional resistance from conservative university IT departments.

4.4 Entity Relationship Diagram (ERD)

The ERD below defines all the core data entities required for the EDYGUIDE system, their attributes, and the relationships between them. These entities map directly to Firestore collections in the database layer.

Required Entities

Entity	Key Attributes	Relationships
Student	student_id (PK), name, email, password_hash, university_id, department, level, GPA, attendance_pct, mental_health_score, created_at	Has many: ChatSessions, ServiceRequests, Payments, TutoringBookings, Applications
Course	course_id (PK), name, code, instructor_id (FK), department, credit_hours, semester, schedule_json	Belongs to: Instructor. Has many: Enrollments, FAQs
Enrollment	enrollment_id (PK), student_id (FK), course_id (FK), grade, attendance_count, status	Belongs to: Student, Course
ChatSession	session_id (PK), student_id (FK), started_at, last_active, context_tag	Belongs to: Student. Has many: ChatMessages
ChatMessage	message_id (PK), session_id (FK), sender (student bot), content, timestamp, intent_tag	Belongs to: ChatSession
ServiceRequest	request_id (PK), student_id (FK), service_id (FK), status, submitted_at, resolved_at	Belongs to: Student, Service. Has many: RequestMessages
RequestMessage	msg_id (PK), request_id (FK), sender_id (FK), sender_role (student admin), content, timestamp	Belongs to: ServiceRequest

PROJECT : Student Service Chatbot

Service	service_id (PK), name, category, description	Has many: FAQs, ServiceRequests
FAQ	faq_id (PK), service_id (FK), question, answer, language (AR EN)	Belongs to: Service
Payment	payment_id (PK), student_id (FK), amount, type (tuition fine), method (Fawry VodafoneCash), status, fawry_ref, timestamp	Belongs to: Student
CareerProfile	profile_id (PK), student_id (FK), cv_url, ats_score, linkedin_url, target_roles_json, skills_json, last_updated	Belongs to: Student. Has many: InternshipApplications
InternshipApplication	app_id (PK), profile_id (FK), company, role, status, applied_at	Belongs to: CareerProfile
WellnessLog	log_id (PK), student_id (FK), mood_score (1-5), notes, timestamp, escalated (bool), counselor_id (FK nullable)	Belongs to: Student
Scholarship	scholarship_id (PK), title, provider, amount, eligibility_json, deadline, status (open closed)	Has many: ScholarshipApplications
ScholarshipApplication	app_id (PK), student_id (FK), scholarship_id (FK), status, submitted_at	Belongs to: Student, Scholarship
TutoringListing	listing_id (PK), tutor_id (FK student), course_id (FK), availability_json, rating_avg	Belongs to: Student (as tutor), Course. Has many: TutoringBookings
TutoringBooking	booking_id (PK), listing_id (FK), student_id (FK), Scheduled_at, Status,	Belongs to: TutoringListing, Student

PROJECT : Student Service Chatbot

	rating	
Admin	admin_id (PK), Name, Email, Role, university_id	Has many: RequestMessages, Scholarships (managed)

Key Cardinality Rules

The following cardinality rules govern the relationships between entities and define how data is structured and queried across the system:

- Student → ChatSession: One-to-Many. A student can have unlimited chat sessions; each session belongs to exactly one student.
- ChatSession → ChatMessage: One-to-Many. A session contains many messages in chronological order.
- Student → Enrollment → Course: Many-to-Many resolved via the Enrollment junction entity. A student enrolls in many courses; each course has many enrolled students.
- Student → ServiceRequest → Service: Many-to-Many resolved via ServiceRequest. Each request links one student to one service type.
- ServiceRequest → RequestMessage: One-to-Many. An admin and student exchange messages within a single service request thread.
- Student → WellnessLog: One-to-Many. Each mood check-in creates a new log entry with an optional escalation flag.
- TutoringListing → TutoringBooking: One-to-Many. A tutor's listing can receive many bookings from different students.
- Scholarship → ScholarshipApplication: One-to-Many. Many students can apply for the same scholarship.
- CareerProfile → InternshipApplication: One-to-Many. A student's career profile can have many internship applications over time.

PROJECT : Student Service Chatbot

Section 5: Connecting Frontend, Backend & Database

This section explains exactly how the three technical layers — Flutter (Frontend), FastAPI (Backend), and Firebase Firestore (Database) — are wired together into a functioning, deployable system. Following these steps will allow the development team to complete the project end-to-end.

5.1 Overview: How the Three Layers Communicate

The three layers follow a standard client-server architecture with a clear separation of concerns:

Layer	Responsibility	Talks To
Flutter (Frontend)	Renders UI, captures user input, displays data fetched from the backend. Never touches the database directly.	Backend only
FastAPI (Backend)	Processes business logic, authenticates users, calls AI engines, talks to Firestore, calls Fawry/Vodafone Cash APIs.	Frontend + Database
Firestore (Database)	Stores and retrieves all persistent data. Enforces security rules. Streams real-time updates to subscribed backend listeners.	Backend only

5.2 Step-by-Step Integration Guide

Step 1 — Set Up the Backend (FastAPI)

Start with the backend since both Flutter and Firestore depend on it being available.

- Install Python 3.11+ and create a virtual environment: `python -m venv venv`
- Install dependencies: `pip install fastapi uvicorn firebase-admin python-jose[cryptography] httpx`
- Download your Firebase project's service account JSON key from the Firebase Console (Project Settings > Service Accounts).
- Initialize the Firebase Admin SDK in the backend: `import firebase_admin; firebase_admin.initialize_app(credentials.Certificate('serviceAccount.json'))`
- Define your API routers — one per domain (e.g., `routers/academic.py`, `routers/payments.py`, `routers/career.py`).
- Run the backend locally: `uvicorn main:app --reload --port 8000`
- Confirm the auto-generated docs are accessible at `http://localhost:8000/docs` before proceeding.

Step 2 — Connect the Backend to Firestore

PROJECT : Student Service Chatbot

The backend uses the Firebase Admin SDK to read and write to Firestore. The Flutter app never touches Firestore directly.

- In each FastAPI endpoint, import the Firestore client: `db = firebase_admin.firestore.client()`
- To read a student record: `doc = db.collection('students').document(student_id).get()`
- To write a new chat message: `db.collection('chat_messages').add({'session_id': sid, 'content': msg, 'timestamp': firestore.SERVER_TIMESTAMP})`
- To listen for real-time changes (e.g., document workflow updates), use a Firestore snapshot listener in a background thread or Cloud Function.
- Set Firestore Security Rules to allow only the backend's service account to write, and restrict student-level reads to be enforced by the backend JWT layer — not directly by clients.

Step 3 — Set Up Firebase Authentication

Firebase Authentication handles student identity and generates tokens that both Flutter and FastAPI use to verify requests.

- Enable Email/Password and optionally Google SSO in the Firebase Console under Authentication > Sign-in Methods.
- In the Flutter app, use the `firebase_auth` package:
`FirebaseAuth.instance.signInWithEmailAndPassword(email, password)`
- After login, Firebase returns an ID Token (a JWT). Flutter must attach this token to every API request in the Authorization header: `Authorization: Bearer <firebase_id_token>`
- In FastAPI, verify the token on every protected endpoint using the Firebase Admin SDK: `decoded = auth.verify_id_token(token)`. This confirms the student's identity without storing passwords in the backend.

Step 4 — Connect Flutter to the Backend

Flutter communicates with the FastAPI backend exclusively via HTTP REST API calls. It never reads from or writes to Firestore directly.

- Add the `dio` package to `pubspec.yaml` for HTTP requests: `dio: ^5.4.0`
- Create a central API service class in Flutter (`lib/services/api_service.dart`) that handles base URL configuration, header injection (Bearer token), and error handling.
- For each screen, call the relevant backend endpoint. Example — fetching the student's schedule: `final response = await dio.get('/academic/schedule', options: Options(headers: {'Authorization': 'Bearer \">$token'}))`
- Parse the JSON response and update the UI using a state management solution (Provider or Riverpod). Never hard-code data in the UI layer.
- For real-time features (e.g., live chat messages), Flutter polls the backend every 2-3 seconds or uses WebSockets — implement using FastAPI's WebSocket support: `@app.websocket('/ws/chat/{session_id}')`

Step 5 — Connect the AI Engines

PROJECT : Student Service Chatbot

The AI services (Llama-3 for career/CV, SVM for GPA prediction) are called by the backend and their results returned to Flutter — the app never calls AI APIs directly.

- For Llama-3 via Watsonx.ai: configure the API key and endpoint in backend environment variables (.env file). Call the endpoint using httpx in an async FastAPI route.
- For the SVM GPA predictor: train the model offline using scikit-learn on historical anonymized grade data, serialize it with joblib, and load it once at FastAPI startup: `model = joblib.load('models/gpa_predictor.pkl')`
- For the plagiarism checker: integrate with an external API (e.g., Copyleaks or iThenticate) or host an open-source similarity model (e.g., sentence-transformers). The backend calls the engine, receives the report, and returns a structured JSON to Flutter.
- For mental health NLP: deploy a fine-tuned Arabic dialect sentiment model (e.g., CAMEL Tools or AraBERT) as a separate microservice. FastAPI calls this service when a wellness message is received.

Step 6 — Deploy to Production

Once all local integrations are tested, deploy each layer independently.

- Backend: containerize using Docker (`docker build -t edyguide-api .`) and deploy to Google Cloud Run or a university-hosted server. Set environment variables for all API keys and Firebase credentials via Cloud Run secrets.
- Database: Firestore is already cloud-hosted. Export Firestore Security Rules from local `firebase.rules` and deploy: `firebase deploy --only firestore:rules`
- Frontend: build the Flutter mobile app for Android (`flutter build apk`) and iOS (`flutter build ipa`). Distribute via the university's MDM system or publish to Google Play / Apple App Store.
- Configure CORS in FastAPI to only allow requests from trusted origins (the Flutter app's bundle ID and any admin web portal domain).
- Set up monitoring: use Google Cloud Logging for backend logs, Firebase Crashlytics for Flutter crash reports, and Firebase Performance Monitoring for API response time tracking.

5.3 Data Flow Example: Student Sends a Chat Message (AI Concierge Flow)

The following example traces a single action — a student typing a question into the AI Concierge — through all three layers from the moment the message is sent to the moment the AI response appears on screen:

1	Flutter	Student types 'What is my predicted GPA if I get a B in Calculus?' in the Concierge Chat screen and taps Send. Flutter appends the message to the local chat UI immediately (optimistic update) and sends a POST request to the backend: <code>POST /concierge/message</code> with the message text and current
---	---------	---

PROJECT : Student Service Chatbot

		session_id in the body, plus the Firebase ID Token in the Authorization header.
2	FastAPI	The backend receives the request and calls <code>auth.verify_id_token(token)</code> to confirm the student's identity. It retrieves the student's current GPA, enrolled courses, and credit hours from Firestore (students collection) to use as context for the AI prompt.
3	FastAPI	The backend classifies the intent of the message as 'gpa_prediction' using a lightweight intent classifier. It then constructs a structured prompt for the SVM prediction model, passing the student's current GPA, the Calculus course weight, and the hypothetical grade B as inputs. The model returns a predicted GPA value.
4	FastAPI	The backend saves the student's message and the AI response as two new documents in the chat_messages Firestore collection, both linked to the current session_id. It returns a JSON response to Flutter containing the AI reply text and a timestamp.
5	Flutter	Flutter receives the JSON response from the backend. The chat UI replaces the optimistic placeholder with the confirmed message and renders the AI reply in a new chat bubble: 'Based on your current GPA of 3.4 and the weight of Calculus, scoring a B would bring your GPA to approximately 3.31.'
6	Flutter	If the student sends a follow-up question that requires a free-text LLM response (e.g., 'Which electives should I take to recover my GPA?'), Flutter sends another POST to the backend. This time the backend forwards the full conversation history plus student context to Llama-3 via the Watsonx.ai API, waits for the LLM response, saves it to Firestore, and returns it to Flutter — completing the same cycle.

End of Document

EDYGUIDE — Confidential B2B Technical Specification | Version 1.0